

Smart Hospital Management & Proximity Matching

Bahria University — Department of Software Engineering

A semester project implemented in C++ with Qt6: an object-oriented Hospital Management System that automates patient-to-hospital allocation using proximity matching and intelligent doctor assignment.





Project Overview & Problem Statement

Manual patient allocation and paper-based records create delays, misassignments, and poor utilization of specialist resources. This project introduces an integrated HMS that automates registration, ranks nearby hospitals using graph-based proximity, and persists clinical history—reducing response time and human error.

Core Features



Smart Proximity Matching

BFS-based shortest-path ranking across Islamabad sectors to prioritize closest eligible hospitals.



Auto-Doctor Assignment

Keyword-driven specialization inference maps symptoms to specialist pools (Cardiology, Neurology, etc.).



Patient Registration Workflow

Guided registration UI with validation, demographic capture, and symptom entry for automated routing.



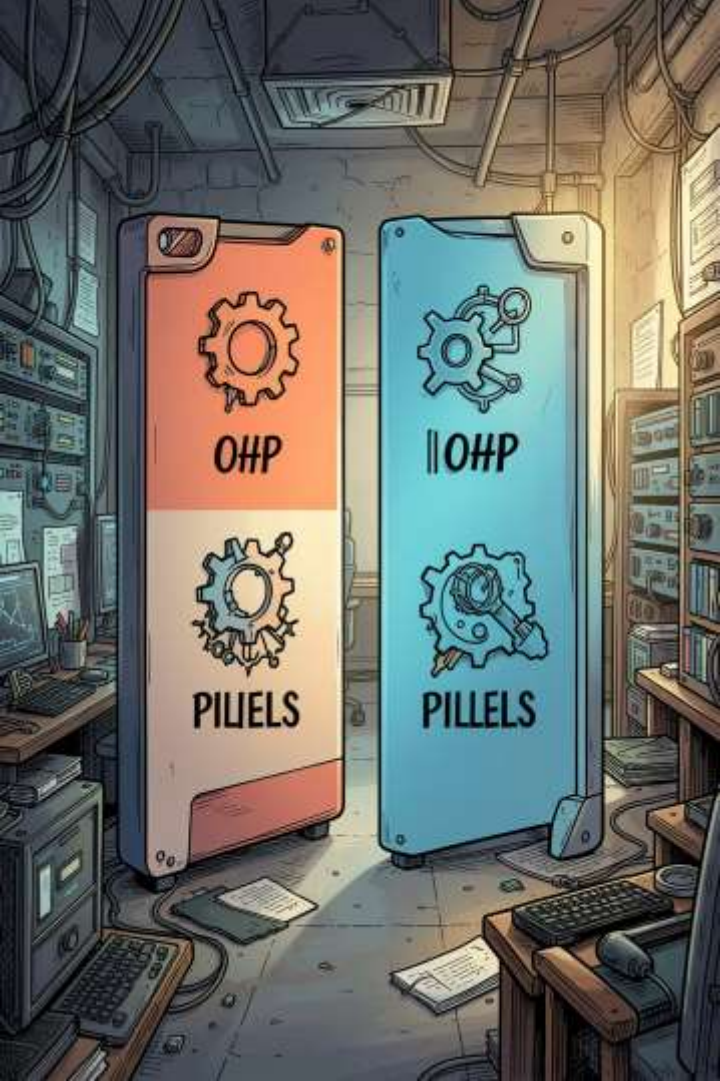
Real-time Dashboard

Live metrics: bed occupancy, active consultations, queued patients, and proximity match outcomes.



SQLite Persistence

Lightweight relational storage for hospitals, doctors, patients, encounters, and audit logs—local, portable, and ACID-compliant for the lab environment.



OOP Architecture — The Four Pillars

Abstraction

Abstract base class
Person declares pure virtual functions (`displayInfo()`, `getIdentifier()`) to define a common interface without implementation details.

Inheritance

Doctor and Patient classes inherit Person, extending behavior with specialization fields (specialty, qualifications) and medical history respectively.

Encapsulation

Private data members with public getters/setters and invariant checks; internal state mutation restricted to class methods to preserve consistency.

Polymorphism

Runtime polymorphism via virtual functions enables unified handling of Person pointers (e.g., UI lists, logs) and dynamic dispatch for `displayInfo()`.

Proximity Matching Algorithm — BFS on Sector Graph

We model Islamabad as an undirected weighted graph of sectors (nodes) with adjacency edges representing traversable connections. To compute proximity ranking:

1. Construct graph $G(V,E)$ from sector connectivity and travel-cost weights.
2. Perform BFS (modified for uniform edge cost) or Dijkstra when weights vary; BFS is used for unweighted sector hops for lab demonstration.
3. Compute shortest path length from patient node to every hospital node and sort hospitals by distance, availability, and specialty match score.

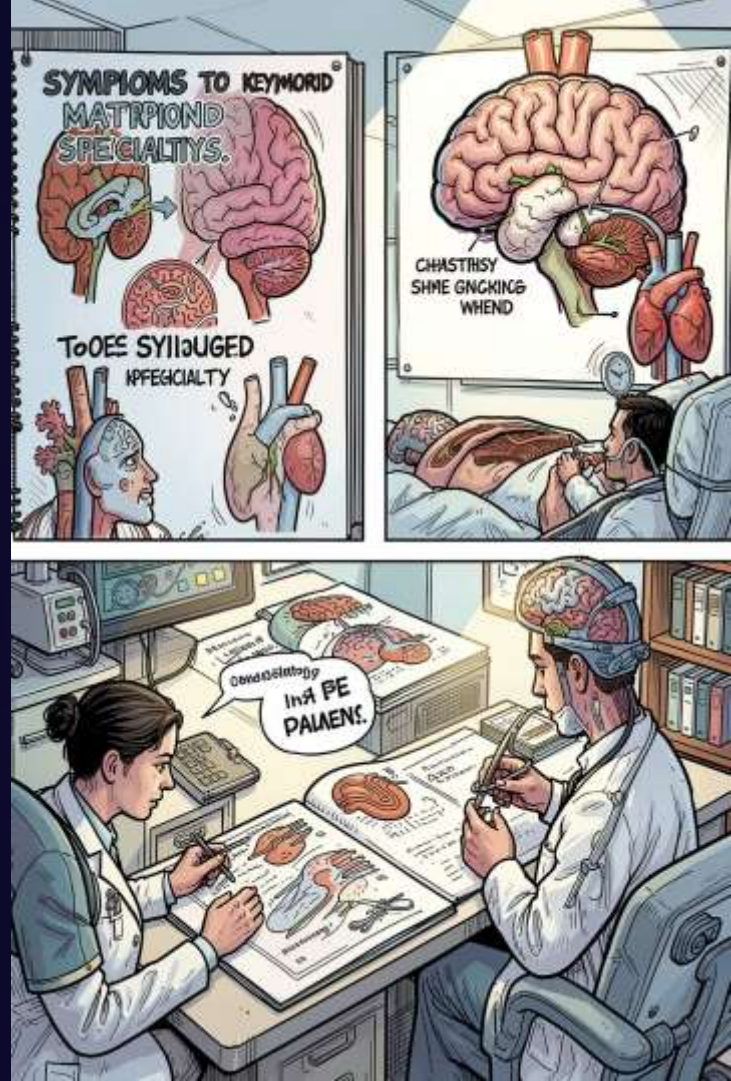
Complexity: BFS runs in $O(|V| + |E|)$. In weighted cases Dijkstra ($O(|E| \log |V|)$) is used; implementation uses adjacency lists and priority queue abstractions.



Auto-Assignment Logic — Symptom → Specialty Mapping

Assignment uses a deterministic keyword-based inference pipeline augmented with rule sets:

- Symptom tokenization and normalization (lowercase, stemming).
- Rule engine maps tokens to specialty weights (e.g., chest pain → Cardiology:0.9, Pulmonology:0.6).
- Combine proximity score and specialty confidence to compute final assignment score: $\text{Score} = \alpha \cdot (1/\text{distance}) + \beta \cdot \text{specialtyConfidence} + \gamma \cdot \text{availability}$.
- Top-ranked doctor/hospital pair is recommended; human override remains available in UI.



Graphical User Interface (Qt6)



The GUI is implemented in Qt6 using Model–View architecture (QAbstractItemModel for patient and hospital tables). Key components:

- Navigation: Sidebar for Modules (Registration, Dashboard, Assignments, Records).
- Real-time cards: custom QWidgets for metrics (occupancy, queue length).
- Dynamic tables: sortable QTableView bound to SQLite via QSqlTableModel for immediate persistence.
- Responsive dialogs: non-blocking assignment dialogs and confirmation flows using signals/slots.

Data Management & Storage — SQLite Integration

Persistence architecture uses a Database Layer encapsulating SQLite operations. Design highlights:

- Schema: hospitals(h_id, name, sector, capacity), doctors(d_id, name, specialty, hospital_id), patients(p_id, name, age, symptoms), encounters(e_id, patient_id, doctor_id, timestamp, notes).
- Access layer: parameterized queries and prepared statements to prevent injection; transactions for multi-step operations (assignment + logging).
- Backup & export: nightly local dump and CSV export for instructor review.





Registration

Proximity
Ranking

Selection

Consultation

Discharge

This diagram summarizes the consultation workflow: patients register → system ranks hospitals via proximity algorithm → recommended hospital/doctor is presented → consultation and notes logged → discharge and persistent encounter record stored in SQLite.

Project Structure & Technical Specs

Folder Organization

- Domain Layer: classes, interfaces (Person, Patient, Doctor, Hospital, Graph).
- Database Layer: SQLite access objects, migrations, schema definitions.
- GUI Layer: Qt widgets, models, views, and controllers.
- Tests & Tools: unit tests (GoogleTest), sample data, scripts.

Technical Stack

- C++17 for language features and modern idioms (smart pointers, move semantics).
- Qt6 and Qt Creator IDE for UI and cross-platform builds.
- MinGW (Windows) or GCC/Clang toolchains for compilation; CMake build system.
- GitHub for version control and CI basics (build checks).

